HOTEL ROOM BOOKING WEB APPLICATION

Project Overview

Elevator Pitch

A hotel room booking web application where visitors browse available rooms, view detailed information (type, capacity, pricing), and authenticated users make reservations with automatic availability checking and pricing. The system handles the full booking lifecycle from discovery through confirmation to cancellation.

Problem Statement

Hotel guests need a convenient way to browse available rooms, check pricing and capacity, and make reservations without manual coordination. The application solves the problem of room discovery and booking management by centralizing room information and automating availability checking, price calculation, and reservation tracking.

Solutions

The platform provides: (1) a room browsing interface showing availability, type, and pricing (F-001, F-002), (2) an authenticated booking flow with date and capacity validation (F-004, F-005, F-006), (3) automatic pricing with long-stay discounts (F-007), (4) booking history and self-service cancellation (F-008, F-009), and (5) a REST API for external client integration (F-012, F-013).

Success Metrics

Number of registered users, daily/weekly bookings, cancellation rate, average booking value, average stay length, room utilization rate, long-stay discount usage rate.

Competitors

Hotel booking platforms, room reservation systems, hotel management software, online hotel booking, accommodation booking.

Vision

A server-rendered Django monolith with a parallel REST API, designed for future SPA/mobile client integration. The service layer supports scalability, with future plans for a dedicated admin interface for room management, payment integration, and notifications.

Target Audience

Unauthenticated Visitor

Needs & Goals: Browse up to 6 available rooms, View full room details (type, capacity, price, description), Register a new account, Log in with existing credentials, Access Home Page (Room Listing), Room Detail Page, Login Page, Registration Page.

Authenticated User

Needs & Goals: All visitor capabilities, Book a room with date and guest selection, View personal booking history with status, Cancel confirmed bookings, Log out, Access Home Page (Room Listing), Room Detail Page, Book Room Page, My Bookings Page.

Key Features

Available Room Browsing

Users can view a grid of up to 6 available hotel rooms on the home page, each displaying room number, type (Single/Double/Suite), guest capacity, and price per night.

Room Detail View

Users can view complete details of any room including type, capacity, price per night, and free-text description.

Context-Aware Navigation

The navigation bar dynamically shows different links based on authentication state — 'My Bookings' and 'Logout' for logged-in users, 'Login' for visitors.

Room Booking with Date Selection

Authenticated users can book a specific room by selecting check-in and check-out dates and specifying guest count, with the system creating a confirmed reservation with a unique UUID confirmation code.

Booking History View

Authenticated users can view all their bookings showing room details, dates, duration in nights, total price, and status (confirmed or cancelled).

Client-Side Date Validation

JavaScript prevents selection of past check-in dates and automatically sets the check-out minimum to one day after the selected check-in date.

Server-Side Booking Validation

The system validates that check-in is not in the past, guest count does not exceed room capacity, the room has no overlapping confirmed bookings, and check-out is after check-in.

Automatic Price Calculation with Long-Stay Discount

Total price is calculated as price_per_night × number of nights, with a 10% discount automatically applied for stays of 7 or more nights.

Booking Cancellation

Users can cancel confirmed bookings with a browser confirmation prompt. Cancelled bookings free up the room for future reservations.

REST API for Room Availability

A REST API endpoint supports listing rooms with optional date-based availability filtering via query parameters, for consumption by external clients.

User Login

Existing users can authenticate with username and password to access booking features.

REST API for Booking Management

REST API endpoints support listing user bookings, creating bookings, and cancelling bookings via JSON, for consumption by external clients.

Phasing

Phase 1: Phase 1: Core Booking Platform

Rationale: This phase delivers the core functionality of the hotel booking platform, allowing users to browse rooms, register, log in, book rooms with validation and automatic pricing, and manage their bookings. This is a launch-able MVP.

Features:

- Available Room Browsing
- Room Detail View
- Context-Aware Navigation
- Room Booking with Date Selection
- Client-Side Date Validation
- Server-Side Booking Validation
- Automatic Price Calculation with Long-Stay Discount
- Booking History View
- Booking Cancellation
- User Registration
- User Login

Phase 2: Phase 2: API Integration

Rationale: This phase focuses on exposing the core booking functionality via REST APIs, enabling integration with external clients like a Single Page Application (SPA) or mobile application. This phase is demo-able and sets the foundation for broader platform reach.

Features:

- REST API for Room Availability
- REST API for Booking Management

User Flows

Visitor Browses Available Rooms

1. 1. User opens the app and lands on the home page (/).
2. 2. App displays a heading 'Available Rooms' followed by a grid of up to 6 room cards, each showing room number, type (Single/Double/Suite), guest capacity, and price per night.
3. 3. If no rooms are available, the app displays the message 'No rooms available.' instead of the grid.
4. 4. The navigation bar shows the 'Hotel' logo, a 'Rooms' link, and a 'Login' link (since the user is not authenticated).
5. 5. User scans the room cards to compare options by type, capacity, and price.

Visitor Views Room Details

1. 1. User is on the home page viewing available room cards.
2. 2. User clicks the 'View' button on a room card.
3. 3. App navigates to the room detail page at /rooms//.
4. 4. App displays the room number as a heading, followed by the room type, guest capacity, price per night, and a free-text description.
5. 5. Since the user is not authenticated, the app displays a 'Login to Book' button instead of 'Book This Room'.
6. 6. User clicks 'Login to Book'.
7. 7. App navigates to the login page at /login/.

User Views Their Bookings

1. 1. User clicks 'My Bookings' in the navigation bar.
2. 2. App navigates to /bookings/my/.
3. 3. System queries all Bookings for the authenticated user with room data pre-fetched.
4. 4. App displays booking cards with room number, type, dates, nights, total price, and status.
5. 5. Confirmed bookings show a red 'Cancel' button; cancelled bookings do not.
6. 6. If no bookings exist, 'No bookings yet.' is displayed.

Visitor Registers a New Account

1. 1. User navigates to the registration page at /register/.
2. 2. App displays a 'Register' heading and a form with three fields: username, email, and password.
3. 3. User enters a username, email address, and password.
4. 4. User clicks the 'Register' button to submit the form.
5. 5. System creates a new User record with a hashed password.
6. 6. System automatically logs the user in by creating a session.
7. 7. User is redirected to the home page (/).
8. 8. The navigation bar now shows 'My Bookings' and 'Logout' links.
9. 9. (Error path: If the username already exists, HTTP 500 is returned due to unhandled IntegrityError.)

User Logs Out

1. 1. User clicks 'Logout' in the navigation bar.
2. 2. System destroys the session.
3. 3. User is redirected to /.
4. 4. Navigation now shows 'Login' instead of 'My Bookings' and 'Logout'.

User Books a Room

1. 1. User is logged in and views the home page (/).
2. 2. App displays up to 6 available rooms in a card grid.
3. 3. User clicks 'View' on a room card.
4. 4. App navigates to /rooms// showing full room details.
5. 5. User clicks 'Book This Room'.
6. 6. App navigates to /bookings/book// displaying the booking form.
7. 7. Client-side JS sets check-in minimum to today.
8. 8. User selects a check-in date.
9. 9. Client-side JS updates check-out minimum to check-in + 1 day.
10. 10. User selects a check-out date and enters guest count.
11. 11. User clicks 'Confirm Booking'.
12. 12. System validates: check-in not in past, guests within capacity, no overlapping confirmed bookings, check-out after check-in.
13. 13. System calculates total price (price_per_night × nights, 10% discount for 7+ nights).
14. 14. System creates Booking with status='confirmed' and UUID confirmation code inside a database transaction.
15. 15. User is redirected to /bookings/my/ with success message 'Booked! Code: '.
16. 16. (Error paths: past check-in, capacity exceeded, unavailable dates, or invalid date range each re-render the form with a specific error message.)

User Cancels a Booking

1. 1. User is on /bookings/my/ viewing their bookings.
2. 2. User clicks the red 'Cancel' button on a confirmed booking.
3. 3. Browser shows confirmation dialog 'Cancel?'.
4. 4. User clicks 'OK'.
5. 5. System verifies booking belongs to user and is not already cancelled.
6. 6. System sets booking.status to 'cancelled' and saves.
7. 7. User is redirected to /bookings/my/ with 'Booking cancelled.' message.
8. 8. The room is now available for the previously booked dates.
9. 9. (Error path: already cancelled → error message 'Booking is already cancelled.')

User Personas

Casual Browser

Role: Any age, Potential Guest

Needs & Goals: Quickly find available rooms, understand hotel offerings, decide if they want to book.

Pain Points: Limited room display (6 rooms), no filtering/search, cannot check specific availability without logging in, registration process lacks robust validation.

Confirmed Guest

Role: Any age, Registered Guest

Needs & Goals: Book rooms efficiently, manage existing reservations, view booking history.

Pain Points: Cannot modify bookings (only cancel), no search/filter on booking history, no notifications, no payment integration, logout uses GET.

Design

Principles

Simplicity & Clarity: Utilize card-based layouts for room browsing and clear form-based interactions for booking and authentication.
Feedback & Validation: Provide immediate feedback through server-side validation messages and client-side date constraints. Use alert banners for success and error notifications.
Contextual Experience: Dynamically adapt UI elements, such as navigation links and booking actions, based on the user's authentication state.
User Control: Implement confirmation dialogs for destructive actions like booking cancellation to prevent accidental data loss.
Foundational Responsiveness: Employ basic responsive design principles using CSS grid for room cards and flexbox for navigation, ensuring usability across different screen sizes, though without explicit mobile breakpoints or hamburger menus.
Semantic Structure: Use semantic HTML forms with CSRF protection for secure and accessible interactions.

Sitemap

- Rooms (/rooms) - Main section for browsing hotel rooms
- Home Page (Room Listing) (/) - Displays up to 6 available rooms in a card grid with type, capacity, and pricing
- Room Detail Page (/rooms//) - Shows full room details including type, capacity, price, and description with booking action
- Bookings (/bookings) - Section for managing user bookings
- Book Room Page (/bookings/book//) - Booking form with check-in/check-out date pickers and guest count
- My Bookings Page (/bookings/my/) - Lists all user bookings with status, pricing, and cancellation action
- Authentication (/auth) - Section for user authentication and registration
- Login Page (/login/) - Username and password login form
- Registration Page (/register/) - New account creation form with username, email, and password
- API (no UI) (/api) - API endpoints without direct UI pages
- Room List API (/api/rooms/) - REST endpoint: lists rooms with optional date-based availability filtering
- My Bookings API (/api/bookings/) - REST endpoint: lists authenticated user's bookings
- Create Booking API (/api/bookings/create/) - REST endpoint: creates a new booking via JSON
- Cancel Booking API (/api/bookings//cancel/) - REST endpoint: cancels a booking via JSON

Navigation Design

The application utilizes a top navigation bar (`top_navbar`) as its primary navigation structure. This bar is defined in `base.html` and dynamically adjusts its links based on the user's authentication state.

Always Visible Links:
- **Hotel (logo):** Links to the Home Page (`/`)
- **Rooms:** Links to the Home Page (`/`)

Authenticated User Links (visible when logged in):
- **My Bookings:** Links to the My Bookings Page (`/bookings/my/`)
- **Logout:** Links to the Logout endpoint (`/logout/`)

Unauthenticated User Links (visible when logged out):
- **Login:** Links to the Login Page (`/login/`)

Responsive Behavior:
The navigation bar is designed using flexbox, ensuring a single-row layout. It uses `system-ui` font. There are no explicit mobile breakpoints or a hamburger menu implemented; the navigation is expected to adapt fluidly based on available space.

User Journeys

User opens the application

Context: An unauthenticated visitor opens the application and browses the available hotel rooms displayed on the home page.

Steps:

1. **[User]** User opens the app and lands on the home page (/).
2. **[System]** App displays a heading 'Available Rooms' followed by a grid of up to 6 room cards, each showing room number, type (Single/Double/Suite), guest capacity, and price per night.
3. **[System]** If no rooms are available, the app displays the message 'No rooms available.' instead of the grid.
4. **[System]** The navigation bar shows the 'Hotel' logo, a 'Rooms' link, and a 'Login' link (since the user is not authenticated).
5. **[User]** User scans the room cards to compare options by type, capacity, and price.

User clicks 'View' on a room card

Context: An unauthenticated visitor navigates from the home page to a room's detail page to view full information, then is directed to log in before booking.

Steps:

1. **[User]** User is on the home page viewing available room cards.
2. **[User]** User clicks the 'View' button on a room card.
3. **[System]** App navigates to the room detail page at /rooms//.
4. **[System]** App displays the room number as a heading, followed by the room type, guest capacity, price per night, and a free-text description.
5. **[System]** Since the user is not authenticated, the app displays a 'Login to Book' button instead of 'Book This Room'.
6. **[User]** User clicks 'Login to Book'.
7. **[System]** App navigates to the login page at /login/.

User decides to create a new account

Context: A new visitor creates an account by filling in the registration form, is automatically logged in, and is redirected to the home page.

Steps:

1. **[User]** User navigates to the registration page at /register/.
2. **[System]** App displays a 'Register' heading and a form with three fields: username, email, and password.
3. **[User]** User enters a username, email address, and password.
4. **[User]** User clicks the 'Register' button to submit the form.
5. **[System]** System creates a new User record with a hashed password.
6. **[System]** System automatically logs the user in by creating a session.
7. **[System]** User is redirected to the home page (/).
8. **[System]** The navigation bar now shows 'My Bookings' and 'Logout' links.
9. **[System]** (Error path: If the username already exists, HTTP 500 is returned due to unhandled IntegrityError.)

User wants to log in

Context: A returning visitor logs into their existing account using their username and password.

Steps:

1. **[User]** User navigates to the login page at /login/.
2. **[System]** App displays a 'Login' heading and a form with username and password fields.
3. **[User]** User enters their username and password.
4. **[User]** User clicks the 'Login' button.
5. **[System]** System authenticates credentials via Django's authenticate function.
6. **[System]** On success, a session is created and user is redirected to /.
7. **[System]** The navigation bar now shows 'My Bookings' and 'Logout'.
8. **[System]** (Error path: Invalid credentials cause re-render with 'Invalid credentials.' message.)

User decides to book a room

Context: An authenticated user browses rooms, views a room's details, fills in the booking form, and confirms a reservation.

Steps:

1. **[User]** User is logged in and views the home page (/).
2. **[System]** App displays up to 6 available rooms in a card grid.
3. **[User]** User clicks 'View' on a room card.
4. **[System]** App navigates to /rooms// showing full room details.
5. **[User]** User clicks 'Book This Room'.
6. **[System]** App navigates to /bookings/book// displaying the booking form.
7. **[System]** Client-side JS sets check-in minimum to today.
8. **[User]** User selects a check-in date.
9. **[System]** Client-side JS updates check-out minimum to check-in + 1 day.
10. **[User]** User selects a check-out date and enters guest count.
11. **[User]** User clicks 'Confirm Booking'.
12. **[System]** System validates: check-in not in past, guests within capacity, no overlapping confirmed bookings, check-out after check-in.
13. **[System]** System calculates total price (price_per_night × nights, 10% discount for 7+ nights).
14. **[System]** System creates Booking with status='confirmed' and UUID confirmation code inside a database transaction.
15. **[System]** User is redirected to /bookings/my/ with success message 'Booked! Code: '.
16. **[System]** (Error paths: past check-in, capacity exceeded, unavailable dates, or invalid date range each re-render the form with a specific error message.)

User wants to view their bookings

Context: An authenticated user navigates to the My Bookings page to review all their reservations.

Steps:

1. **[User]** User clicks 'My Bookings' in the navigation bar.
2. **[System]** App navigates to /bookings/my/.
3. **[System]** System queries all Bookings for the authenticated user with room data pre-fetched.
4. **[System]** App displays booking cards with room number, type, dates, nights, total price, and status.
5. **[System]** Confirmed bookings show a red 'Cancel' button; cancelled bookings do not.
6. **[System]** If no bookings exist, 'No bookings yet.' is displayed.

User wants to cancel a booking

Context: An authenticated user cancels one of their confirmed bookings.

Steps:

1. **[User]** User is on /bookings/my/ viewing their bookings.
2. **[User]** User clicks the red 'Cancel' button on a confirmed booking.
3. **[System]** Browser shows confirmation dialog 'Cancel?'.
4. **[User]** User clicks 'OK'.
5. **[System]** System verifies booking belongs to user and is not already cancelled.
6. **[System]** System sets booking.status to 'cancelled' and saves.
7. **[System]** User is redirected to /bookings/my/ with 'Booking cancelled.' message.
8. **[System]** The room is now available for the previously booked dates.
9. **[System]** (Error path: already cancelled → error message 'Booking is already cancelled.')

User wants to log out

Context: An authenticated user ends their session.

Steps:

1. **[User]** User clicks 'Logout' in the navigation bar.
2. **[System]** System destroys the session.
3. **[System]** User is redirected to /.
4. **[System]** Navigation now shows 'Login' instead of 'My Bookings' and 'Logout'.

Sample Data

users

id	uid	name	email	appr…	prov…	last…	crea…	first…	upd…	sign…	profi…	encr…	allo…
1	user…	User 1	user…	true	email	Last1	2026…	First1	2026…	5	true	hash…	false
2	user…	User 2	user…	false	email	Last2	2026…	First2	2026…	10	false	hash…	false

organizations

id	name	created_at	updated_at
1	Organization 1	2026-03-18T18:29:37.202454+00:00	2026-03-18T18:29:37.202454+00:00
2	Organization 2	2026-03-18T18:29:37.202454+00:00	2026-03-18T18:29:37.202454+00:00

roles

id	name	created_at	updated_at
1	Role 1	2026-03-18T18:29:37.202461+00:00	2026-03-18T18:29:37.202461+00:00
2	Admin Role 2	2026-03-18T18:29:37.202461+00:00	2026-03-18T18:29:37.202461+00:00

users_roles

id	role_id	user_id	created_at	updated_at	organization_id
1	1	1	2026-03-18T18:29:3…	2026-03-18T18:29:3…	1
2	2	2	2026-03-18T18:29:3…	2026-03-18T18:29:3…	1

rooms

Showing first 5 of 6 rows

number	capacity	room_type	created_at	updated_at	description	is_available	organizati…	price_per_…
101	1	single	2026-03-01…	2026-03-01…	Cozy single…	true	1	85.0
102	2	double	2026-03-02…	2026-03-02…	Comfortabl…	true	1	140.0
201	3	double	2026-03-03…	2026-03-03…	Spacious d…	true	1	150.0
202	4	suite	2026-03-04…	2026-03-04…	Executive s…	true	1	320.0
301	1	single	2026-03-05…	2026-03-05…	Quiet singl…		1	95.0

bookings

status	room_id	user_id	check_in	check_…	created…	update…	total_pr…	guests…	organiz…	confirm…
confirmed	1	1	2026-04…	2026-04…	2026-03…	2026-03…	255.0	1	1	550e84…
cancelled	2	2	2026-05…	2026-05…	2026-03…	2026-03…	882.0	2	1	660e84…

User Stories

Browse Available Rooms

Phase 1: Core Booking Platform | Available Room Browsing

As a visitor, **I want** to see a list of available hotel rooms on the home page, **so that** I can browse what is currently offered

Acceptance Criteria:

- The home page displays a grid of up to 6 available rooms.
- Each room card shows the room number, type, guest capacity, and price per night.
- If no rooms are available, a 'No rooms available' message is displayed.
- Each room card has a 'View' button to see more details.

View Room Details

Phase 1: Core Booking Platform | Room Detail View

As a visitor, **I want** to view the full details of a room, **so that** I can decide whether to book it

Acceptance Criteria:

- The room detail page displays the room number, type, capacity, price per night, and a free-text description.
- The page loads successfully for a valid room ID.
- If the room ID is invalid, a 404 error is returned.

Visitor Navigation Links

Phase 1: Core Booking Platform | Context-Aware Navigation

As a visitor, **I want** to see 'Login' in the navigation bar, **so that** I can log in or register

Acceptance Criteria:

- The navigation bar displays a 'Login' link when I am not logged in.
- The 'Login' link navigates to the login page.
- The 'Hotel' logo and 'Rooms' link are always visible.

Authenticated User Navigation Links

Phase 1: Core Booking Platform | Context-Aware Navigation

As a authenticated user, **I want** to see 'My Bookings' and 'Logout' links in the navigation, **so that** I can access my bookings or sign out

Acceptance Criteria:

- The navigation bar displays 'My Bookings' and 'Logout' links when I am logged in.
- The 'My Bookings' link navigates to my booking history page.
- The 'Logout' link logs me out and redirects me to the home page.
- The 'Hotel' logo and 'Rooms' link are always visible.

Book a Room with Dates and Guests

Phase 1: Core Booking Platform | Room Booking with Date Selection

As a authenticated user, **I want** to book a specific room by selecting check-in and check-out dates and specifying guest count, **so that** I can reserve my stay

Acceptance Criteria:

- I can select a check-in date and a check-out date on the booking form.
- I can specify the number of guests for the booking.
- Upon successful submission, a confirmed reservation is created with a unique confirmation code.
- I am redirected to my booking history page with a success message after booking.

Prevent Past Check-in Dates

Phase 1: Core Booking Platform | Client-Side Date Validation

As a authenticated user, **I want** the check-in date picker to prevent selection of past dates, **so that** I cannot book a room starting in the past

Acceptance Criteria:

- The check-in date picker prevents me from selecting a date in the past.
- The minimum selectable check-in date is set to today's date by client-side JavaScript.
- If I try to manually enter a past date, the form should indicate an error or reset the date.

Adjust Check-out Date Minimum

Phase 1: Core Booking Platform | Client-Side Date Validation

As a authenticated user, **I want** the check-out date picker to automatically adjust its minimum date when I change the check-in date, **so that** I cannot select a check-out on or before the check-in

Acceptance Criteria:

- When I select a check-in date, the check-out date picker's minimum selectable date is automatically updated to one day after the check-in date.
- I cannot select a check-out date that is on or before the check-in date.
- The client-side JavaScript handles the dynamic adjustment of the check-out date minimum.

Server-Side Booking Validation

Phase 1: Core Booking Platform | Server-Side Booking Validation

As a authenticated user, **I want** the system to validate my booking details (dates, guest count, room availability) on the server, **so that** I only make valid reservations

Acceptance Criteria:

- The system prevents booking if the check-in date is in the past.
- The system prevents booking if the guest count exceeds the room's capacity.
- The system prevents booking if there are overlapping confirmed bookings for the selected room and dates.
- The system prevents booking if the check-out date is on or before the check-in date.
- If any validation fails, an appropriate error message is displayed on the booking form.

Automatic Price Calculation with Discounts

Phase 1: Core Booking Platform | Automatic Price Calculation with Long-Stay Discount

As a authenticated user, **I want** the system to automatically calculate the total price for my booking, including any applicable long-stay discounts, **so that** I know the final cost

Acceptance Criteria:

- The total price for the booking is calculated as the price per night multiplied by the number of nights.
- A 10% discount is automatically applied if the stay is 7 or more nights.
- The calculated total price is displayed to me before I confirm the booking.
- The final price is quantized to two decimal places.

View My Booking History

Phase 1: Core Booking Platform | Booking History View

As a authenticated user, **I want** to view all my bookings showing room details, dates, duration, price, and status, **so that** I can review my reservation history

Acceptance Criteria:

- I can access a page that lists all my past and current bookings.
- Each booking entry displays the room details (number, type), check-in and check-out dates, duration in nights, total price, and current status (confirmed or cancelled).
- Bookings are displayed in a clear and organized manner.
- If I have no bookings, a message indicating this is displayed.

Cancel a Confirmed Booking

Phase 1: Core Booking Platform | Booking Cancellation

As a authenticated user, **I want** to cancel a confirmed booking, **so that** I can free up the room and withdraw my reservation

Acceptance Criteria:

- I can see a 'Cancel' option for my confirmed bookings in the booking history view.
- When I click 'Cancel', a browser confirmation prompt asks me to confirm the cancellation.
- Upon confirmation, the booking status changes to 'cancelled'.
- The cancelled room becomes available again for new reservations.
- I receive a success message after a successful cancellation.
- I cannot cancel a booking that is already cancelled.

Register a New Account

Phase 1: Core Booking Platform | User Registration

As a new visitor, **I want** to create an account with username, email, and password, **so that** I can access booking features

Acceptance Criteria:

- I can access a registration form with fields for username, email, and password.
- Upon successful submission, a new user account is created.
- I am automatically logged in after successful registration.
- I am redirected to the home page after successful registration.
- The system should handle duplicate usernames gracefully (though current implementation results in 500, a future improvement).

Log In to My Account

Phase 1: Core Booking Platform | User Login

As a existing user, **I want** to log in with my username and password, **so that** I can access authenticated features

Acceptance Criteria:

- I can access a login form with fields for username and password.
- Upon successful submission with valid credentials, I am logged into my account.
- I am redirected to the home page after successful login.
- If I provide invalid credentials, an error message 'Invalid credentials.' is displayed.

Query Room Availability via API

Phase 2: API Integration | REST API for Room Availability

As a external client developer, **I want** to query available rooms via a REST API, including date-based filtering, **so that** I can build custom room browsing experiences

Acceptance Criteria:

- The API endpoint /api/rooms/ returns a list of available rooms.
- I can filter rooms by check-in and check-out dates using query parameters.
- The API response includes room number, type, capacity, and price per night.
- The API supports standard HTTP methods for retrieval (GET).

Manage Bookings via REST API

Phase 2: API Integration | REST API for Booking Management

As a external client developer, **I want** to list, create, and cancel bookings via REST API endpoints, **so that** I can integrate booking functionality into my application

Acceptance Criteria:

- I can retrieve a list of a user's bookings via the /api/bookings/ endpoint.
- I can create a new booking by sending a POST request to /api/bookings/create/ with booking details (room ID, dates, guests).
- I can cancel an existing booking by sending a POST request to /api/bookings/<pk>/cancel/.
- The API endpoints return appropriate JSON responses for success and error scenarios.
- Authentication is required for booking management API endpoints.

Market Sizing

Total Addressable Market (TAM)

Estimated Value: $500,000,000,000.0

Total Addressable Market for the Global Online Hotel Booking Market, encompassing all online platforms and direct hotel websites where consumers can reserve accommodation.

Serviceable Available Market (SAM)

Estimated Value: $75,000,000,000.0

Serviceable Available Market for Direct Hotel Booking Platforms and Independent Hotel Reservation Systems. This segment focuses on solutions that enable individual hotels or smaller chains to manage and facilitate direct online bookings, or new direct-to-consumer platforms offering a streamlined booking experience.

Serviceable Obtainable Market (SOM)

Estimated Value: $2,250,000,000.0

Serviceable Obtainable Market, representing the realistic market share a new, streamlined hotel room booking web application can capture within the Direct Hotel Booking Platforms and Independent Hotel Reservation Systems market over a 5-year horizon.

Key Assumptions

Customer type: Individual Travelers/Guests (end-users making bookings)

Time horizon years: 5

Pricing model basis: Per booking transaction fee or commission (future state, assuming payment integration is added) / Per night rate for rooms.

Avg revenue per user per year: 800.0

Expected market penetration percent: 3.0

Industry category: Online Travel & Hospitality Technology

Geographic scope: Global

Industry growth rate cagr percent: 8.0

Key competitors: ["Hotel booking platforms", "room reservation systems", "hotel management software", "online hotel booking", "accommodation booking"]

Advantage summary: Streamlined user experience for browsing and booking, automated availability and pricing with long-stay discounts, self-service booking management (history, cancellation), and a scalable REST API for future external client integration (SPA/mobile).